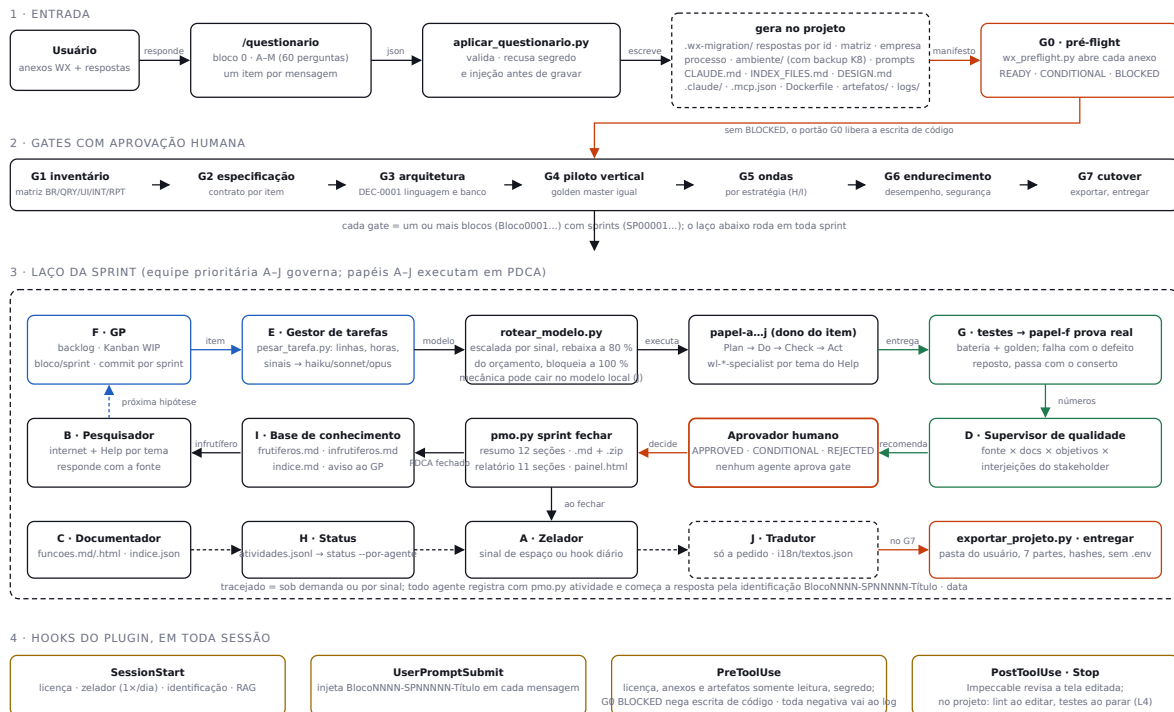


Fluxo atual do WX Claude Code

Versão 3.28.0. Da esquerda para a direita: o que o usuário entrega, o que o questionário gera, os gates com aprovação humana e o laço de sprint da equipe. Os hooks do plugin correm por baixo de tudo. Cada caixa é um agente, um script ou um arquivo que existe no repositório.



Corpus WLanguage 12k (verificado por hash, consultado por tema) e references/ do plugin são a fonte técnica de todos os agentes; regra de negócio só vem do legado com origem localizável.

Built to convert. Engineered to prove.

O plugin em três movimentos: o questionário produz o contexto da primeira sessão; os gates avançam só com o aprovador humano; dentro de cada sprint o GP distribui, o Gestor de tarefas pesa e escolhe o modelo, os papéis executam em PDCA, testes e prova real medem, o Supervisor recomenda e o humano decide. Cada ciclo infrutífero aciona o Pesquisador, e o resultado da sprint sai em relatório, painel, cópia zipada e, no fim, na pasta do usuário.

— fluxo principal — portão ou decisão humana — distribuição do GP — verificação

--- sob demanda ou por sinal

O que roda antes de cada commit

Duas provas, não uma. A bateria confere cada peça isolada; o teste de fluxo confere a **ligação** entre elas — que é onde os defeitos deste projeto sempre apareceram.

PROVA	TAMANHO	O QUE PEGA
tests/testes.py	59 testes	cada peça isolada: validação, hooks, licença, PMO, roteador, artefatos, páginas em dia
tests/fluxo.py	13 passos	o caminho inteiro num projeto novo: questionário → contexto → G0 → artefato → PDF → PMO → roteador → RAG → entrega → registro
validate_plugin_bundle.py --strict	1 comando	arquivos obrigatórios, manifesto, e roda a bateria por dentro
claude plugin validate	1 comando	o manifesto pelos olhos do próprio Claude Code

O teste de fluxo mede cada passo e separa **código esperado de falha**: o G0 devolve 2 para **CONDITIONAL** por contrato, e a negativa do hook no passo 6 é ele fazendo o que deve. Confundir os dois manda alguém caçar um defeito que não existe.

As sete etapas, do install à entrega

O diagrama acima mostra o mecanismo; a lista abaixo, a ordem em que um projeto passa por ele na 3.22.0.

1 Instalar e ativar `claude plugin install`

licenca	serial confere a máquina; sem ele, os scripts do plugin recusam	comandos	o índice: 19 comandos e os 60 ids das perguntas
----------------	---	-----------------	---

2 Perguntar `/wx-claude-code:questionario`

bloco 0	empresa, diretores, logotipos, prazo, orçamento, riscos, GitHub, aprovador (0.1–0.16)	A–E	evidências: SQL da análise e os quatro PDFs
F	tela modelo e estilo (F0–F13)	G	corpus do Help WLanguage (12k)
		J	economia de tokens

H · I	destino: qualquer linguagem; perfil «outra» é aceito	
K	ambiente (K0–K7) e backup e replicação (K8)	L
M	artefatos do cliente, um por vez, com onde usar	kickoff, hooks, MCP, implantação, esqueleto de ERP (L1–L6)

3 Gerar o contexto

`aplicar_questionario.py`

CLAUDE.md · INDEX_FILES.md	o que a primeira sessão lê antes de qualquer comando	respostas_questionario.md	as 60 respostas com índice por id, para os agentes
esqueleto de ERP	AGENTS, CONTEXT, ADRs, domínios, database, src, tests, workflows	ambiente e backup	instalador, papéis SQL, .env.exemplo, plano de backup
102 arquivos	nada é sobrescrito ao reaplicar		

4 Provar a entrada

`/wx-claude-code:preflight – G0`

inventário	cada evidência com hash e classificação	relatório	o hook portão_g0 falha fechado se não conseguir lê-lo
PDF → markdown	página, hash e OCR marcado em vez de inventado		

5 Converter

`/wx-claude-code:converter – G1 a G7`

G1 inventário	BR-, QRY-, UI-, RPT-, INT-, DB- com origem localizável	G2 arquitetura	o que cada peça vira; a estratégia escolhida em H

G3 dados	esquema migrado, chaves e integridade	G4 piloto	uma vertical inteira, com golden master
G5 ondas	módulo a módulo, telas pelo Impeccable	G6 homologação	paralelo com o legado
G7 virada	só com backup do legado restaurado, não copiado		

6 Governar

/wx-claude-code:pmo

sprints	BlocoNNNN-SPNNNN, cada uma em .md zipado	PDCA	frutíferos e infrutíferos na base de conhecimento
relatório	onze seções, todo número saindo de arquivo	equipe A–J	zelador, pesquisador, documentador, qualidade, tarefas, GP, testes, status, base, tradutor

7 Entregar

/wx-claude-code:exportar

sete pastas	manifesto com SHA-256 de cada arquivo	sem segredo	recusa .env e arquivo com cara de token
registro	toda operação em .wx-migration/logs/		

As guardas, que rodam sozinhas

Regra que depende de alguém lembrar não é regra. Estas rodam nos eventos do Claude Code, e cada negativa entra no registro de operações.

GUARDA	EVENTO	O QUE FAZ VALER
portão G0	PreToolUse	nenhuma escrita fora de .wx-migration antes do G0 passar; falha fechado
guarda de anexos	PreToolUse	inputs/ e artefatos/ são somente leitura; redirecionamento confere o alvo

GUARDA	EVENTO	O QUE FAZ VALER
filtro de segredos	PreToolUse	token em arquivo ou em linha de comando é recusado
licença	PreToolUse · SessionStart	serial confere a máquina antes de rodar script do plugin
sincronizar PMO	PostToolUse	Kanban acompanha a matriz sem ninguém digitar
identificação	UserPromptSubmit	BlocoNNNN-SPNNNNN-Título · data em toda resposta
RAG	UserPromptSubmit	injeta o trecho mais próximo com arquivo#linha, e o tema do Help
zelador	SessionStart	limpa temporário uma vez por dia e mede o que liberou
impeccable	PostToolUse · Stop	revisão de design da tela que acabou de mudar

O que roda antes de cada commit

Duas provas, não uma. A bateria confere cada peça isolada; o teste de fluxo confere a **ligação** entre elas — que é onde os defeitos deste projeto sempre apareceram.

PROVA	TAMANHO	O QUE PEGA
tests/testes.py	59 testes	cada peça isolada: validação, hooks, licença, PMO, roteador, artefatos, páginas em dia
tests/fluxo.py	13 passos	o caminho inteiro num projeto novo: questionário → contexto → G0 → artefato → PDF → PMO → roteador → RAG → entrega → registro
validate_plugin_bundle.py --strict	1 comando	arquivos obrigatórios, manifesto, e roda a bateria por dentro
claude plugin validate	1 comando	o manifesto pelos olhos do próprio Claude Code

O teste de fluxo mede cada passo e separa **código esperado de falha**: o G0 devolve 2 para **CONDITIONAL** por contrato, e a negativa do hook no passo 6 é ele fazendo o que deve. Confundir os dois manda alguém caçar um defeito que não existe.

Onde as coisas caem

`.wx-migration/` é a governança (respostas, matriz, PMO, ambiente, prompts, logs, extraídos); `inputs/` é a evidência do legado, somente leitura; `artefatos/` é o que o cliente mandou por fora, também somente leitura e com hash; a raiz do projeto recebe o que a primeira sessão lê e o esqueleto de ERP. O organograma de arquivos detalha os 102 arquivos que o questionário pode gerar.

O que roda antes de cada commit

Duas provas, não uma. A bateria confere cada peça isolada; o teste de fluxo confere a **ligação** entre elas — que é onde os defeitos deste projeto sempre apareceram.

PROVA	TAMANHO	O QUE PEGA
<code>tests/testes.py</code>	59 testes	cada peça isolada: validação, hooks, licença, PMO, roteador, artefatos, páginas em dia
<code>tests/fluxo.py</code>	13 passos	o caminho inteiro num projeto novo: questionário → contexto → G0 → artefato → PDF → PMO → roteador → RAG → entrega → registro
<code>validate_plugin_bundle.py --strict</code>	1 comando	arquivos obrigatórios, manifesto, e roda a bateria por dentro
<code>claude plugin validate</code>	1 comando	o manifesto pelos olhos do próprio Claude Code

O teste de fluxo mede cada passo e separa **código esperado de falha**: o G0 devolve 2 para `CONDITIONAL` por contrato, e a negativa do hook no passo 6 é ele fazendo o que deve. Confundir os dois manda alguém caçar um defeito que não existe.